군-학 연계 프로그램 범용 교육

컴퓨터 비전 — YOLO 객체 탐지, 영상 속 표적 찾기

서왕덕

경기대학교 AI컴퓨터공학부 5차시

26.10.14



목차

1. 실습 준비: Colab 열기와 GPU 켜기
2. 복습과 로드맵
3. 1부. 사진 속 물체 찾기: 상자, 이름, 확신도
4. 2부. 영상 속 표적 찾기: 영상은 사진의 연속
5. 3부. YOLO는 어떻게: 한 번만 보고, 격자로 나눈다
6. 4부. 내 데이터로 학습: 서명을 찾는 모델 만들기
7. 정리



실습 준비

Colab 열기 · GPU 켜기 · 셀 1 미리 실행

오늘의 도구 3개



Colab + T4 GPU

3차시에 만든 Google 계정으로. 오늘은 **GPU를 켠다**. 설치할 것 없음



YOLO (yolo26n)

사진 속 물체를 찾는 공개 모델. 노트북이 알아서 내려받는다



makesense.ai

브라우저에서 사진에 상자를 그리는 라벨링 도구. 가입 · 설치 없음

왜 오늘은 GPU인가

3차시 RAG는 CPU로도 됐다. 오늘은 영상 450장을 처리하고 모델을 **학습**시킨다. 4차시에서 본 대로 딥러닝의 계산은 GPU가 수십 배 빠르다. Colab 무료 계정은 T4 GPU 한 장을 빌려 준다 (보장은 아님).

시작 전에 확인

Google 계정 로그인이 돼 있는가(3차시 계정). 비밀번호를 잊었으면 지금 손을 든다. 개인정보가 있는 사진은 오늘 올리지 않는다.

Colab 열기와 노트북 올리기 (3차시 복습)



순서

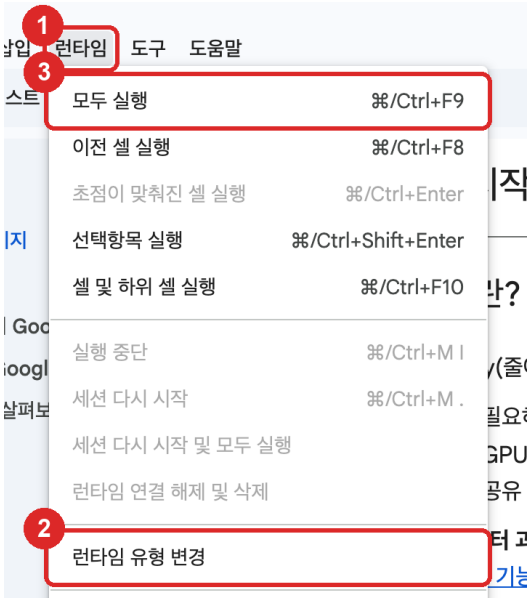
1) 메일 첨부 `yolo_detect.ipynb` 를 **바탕화면에 저장**
2) `colab.research.google.com` 에서 로그인
3) 위 메뉴 (1) **파일** → (2) **노트 업로드** → 바탕화면의 파일 선택
4) 제목이 `yolo_detect.ipynb` 이고 옆에 "모든 변경사항이 저장됨"이 보이면 내 Drive에 들어간 것

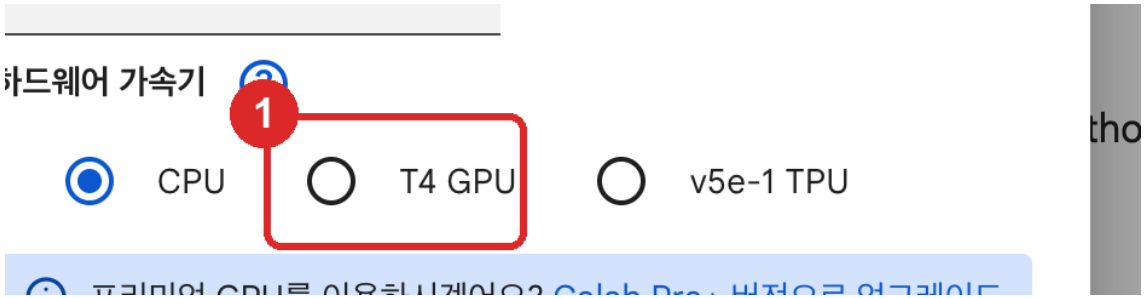
노트북 파일

수업 전 메일로 받은 `yolo_detect.ipynb`. 배포 zip의 `notebook/` 폴더에도 같은 파일이 있다. 메일을 못 받았으면 옆 사람 USB의 zip에서 꺼낸다.

⚠ **3차시와 다른 점**: 링크로 연 남의 노트북은 "Drive에 사본 저장"이 필요했지만, **노트 업로드**로 올린 파일은 처음부터 내 Drive에 있다. 사본 저장은 안 해도 된다.

GPU 켜기: 오늘은 필수





순서

1) 위 메뉴 **1** **런타임** → **2** **런타임 유형 변경**
2) 하드웨어 가속기에서 **T4 GPU** 선택 → **저장**
3) 오른쪽 위에 **T4**가 표시되면 된 것

GPU를 못 받았다고 나오면

무료 계정은 GPU가 **보장되지 않는다**. "현재 사용할 수 없음"이 뜨면 1~2분 뒤 다시 시도하고, 그래도 안 되면 손을 든다. CPU로도 셀은 다 돌아가지만 영상 처리와 학습이 5~10배 느려진다 — 그때는 옆 사람 화면을 같이 본다.

실습 0-1: Colab + GPU 체크리스트 (6분)

prompts/01_colab_gpu_setup.txt — 순서 전문과 안 될 때 표

하나씩 눈으로 확인

✓ Google 계정으로 로그인했고 오른쪽 위에 프로필 아이콘이 보인다
✓ 노트북 제목이 yolo_detect.ipynb 이고 옆에 **"모든 변경사항이 저장됨"** (파일 → 노트 업로드)
✓ 런타임 유형이 **T4 GPU**다 (런타임 → 런타임 유형 변경)
✓ 브라우저의 "비밀번호 저장" 제안은 **거절**했다(공용 PC)

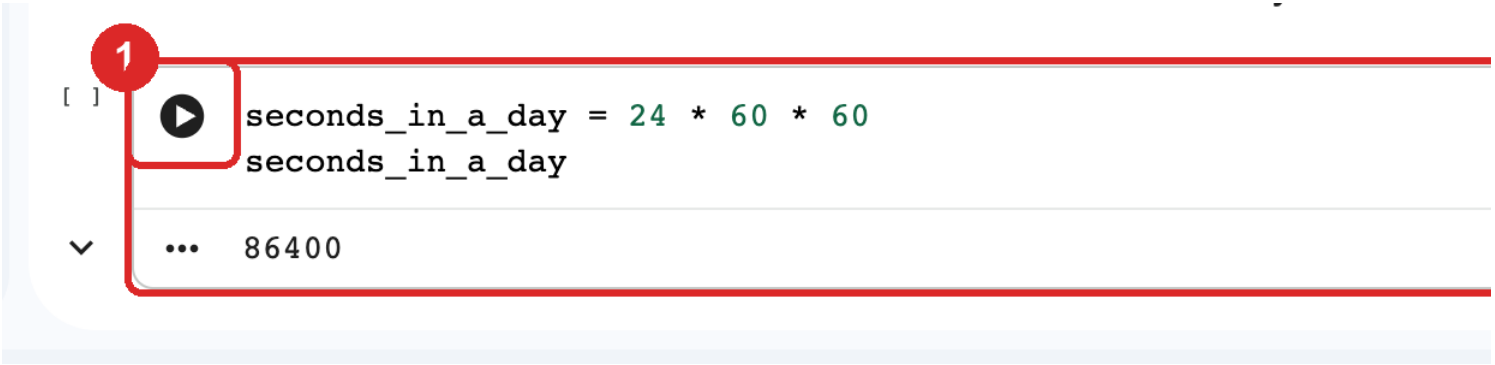
5분 규칙

5분 넘게 막히면 손을 든다. 로그인 · 업로드 · GPU 세 가지는 옆에서 화면을 한 번 보는 것이 가장 빠르다.

먼저 끝난 사람

다음 슬라이드의 **셀 1**을 바로 누른다. 1~2분 걸리는 설치를 강의 중에 미리 끝내 두는 것이다.

실습 0-2: 셀 1 미리 실행 (2분, 기다리는 동안 강의 시작)



▶ 셀 1 왼쪽의 ▶를 누른다 — 이렇게 나오면 정상

첫 줄에 GPU: Tesla T4. 이어서 회색 글씨가 한참 흐른다(도구 설치 · 모델 2개 · 사진 · 영상 · 서명 데이터셋 내려받기). 마지막 줄에 **준비 완료**. 1~2분.

⚠ GPU 없음이 나오면

런타임 유형을 T4로 바꾸지 않은 것. 앞 슬라이드대로 바꾸면 노트북이 **다시 시작**되므로 셀 1을 한 번 더 누른다.

⧗ 기다리는 동안

탭을 닫지 않는다. 다음 섹션(복습 · 로드맵)을 듣는다. 끝나면 셀 왼쪽 괄호에 숫자 [1]이 생긴다.



복습과 로드맵

읽는 AI에서 보는 AI로

오늘 수업 로드맵 (120분)

시간	구간	내용	실습
12분	실습 준비	Colab · 노트 업로드 · T4 GPU · 셀 1 미리 실행	실습 0-1 · 0-2
10분	복습 · 로드맵	4차시 연결 · 세 질문 · 결과물 · 준비 원칙	—
24분	**1부** 사진 속 물체 찾기	분류와 탐지 · 상자 · 이름 · 확신도 · 80종 · 분할	실습 1∼3 · 3b
15분	**2부** 영상 속 표적 찾기	영상은 사진의 연속 · 세기 · 쓰임과 한계	실습 4 · 5
15분	**3부** YOLO는 어떻게	한 번만 본다 · 격자 · 크기와 속도	실습 6
39분	**4부** 내 데이터로 학습	라벨링 · 서명 데이터셋 · 학습 · 전후 비교	실습 7-10
5분	정리	핵심 정리 · 집에서 이어하기 · PC 정리	—

◎ 오늘의 목표 하나

사진과 영상에서 물체를 찾는 공개 모델을 써 보고, 그 모델이 **모르는 물체(서명)** 를 내 데이터로 가르쳐 **학습 전에는 못 찾고 학습 후에는 찾는** 결과를 각자의 Colab에서 만든다. 실습 전문은 `prompts/` 폴더, 00_README부터.

4차시에서 오늘로

4차시: 다이얼
신경망은 수백만 개의 다이얼. 예시를 보여 주면 다이얼이 **스스로** 맞춰진다

4차시: 2012년
사진 백만 장(데이터) + GPU(계산)로 "이 사진은 무엇인가"를 맞히는 분류가 사람을 따라잡았다

오늘: 어디에 있나
무엇인지에 더해 **어디에, 몇 개** 있는지를 찾는다. 같은 다이얼 방식, 질문만 다르다

⇄ 1~3차시와 다른 점

지금까지는 **글**을 다루는 AI(프롬프트 · 스킬 · RAG)였다. 오늘은 **사진과 영상**을 다루는 AI다. 글의 AI가 "다음 단어"를 맞혔다면, 오늘의 AI는 "이 픽셀 뭉치는 무엇이고 어디까지인가"를 맞힌다. 배우는 방식(예시로 다이얼 맞추기)은 똑같다.

↻ 그대로 가져오는 것

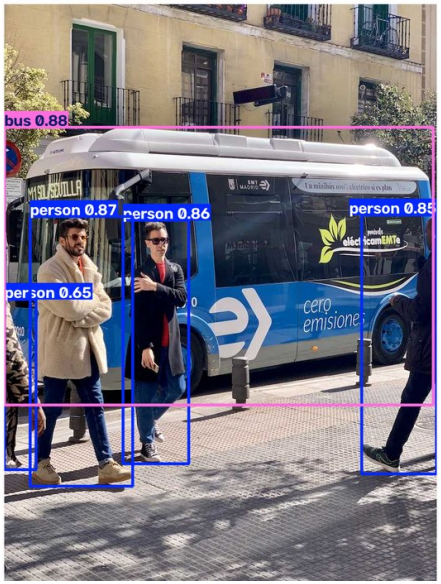
3차시의 Colab 사용법(사본 · ▶ · 바꿔 보세요 줄), 준비 원칙(개인 계정 · 가상 데이터), 4차시의 "데이터 · 계산 · 알고리즘" 세 조각.

보는 AI의 세 가지 질문: 무엇인가 · 어디에 있나 · 어디까지인가

분류: 「버스」



탐지: 상자 + 이름



분할: 픽셀마다 색칠



분류 (Classification)
사진 한 장에 이름 **하나**. "버스". 4차시에서 본 2012년의 문제

탐지 (Detection)
물체마다 **상자 + 이름**. "버스 1, 사람 4, 여기". 오늘의 문제

분할 (Segmentation)
픽셀마다 **어느 물체**인지 색칠. 상자 대신 윤곽. 실습 3b에서 맛본다

같은 사진, 세 가지 답. 비전 AI 문제의 대부분은 이 셋 중 하나이거나 그 조합이다(얼굴 인식 = 탐지 + 분류, 자율주행 = 탐지 + 분할). 질문이 정밀해질수록 답도, 정답을 만드는 일도 커진다.

세 문제 비교: 묻는 것, 답의 모양, 정답 만들기

문제	묻는 것	답의 모양	쓰임 예	정답 만들기
분류	사진이 무엇인가	이름 하나	불량품 양 · 불 판정, 사진 앱 자동 정리	이름 1개
탐지	무엇이 어디에 몇 개	상자, 이름, 확신도	손님 세기, 보행자 감지, 계약서 속 서명	상자 그리기
분할	픽셀마다 어느 물체	색칠된 영역(마스크)	의료 영상 병변 면적, 화상회의 배경 흐리기	윤곽 칠하기 (가장 느림)

풀린 순서

2012년 분류(ImageNet 대회, 4차시) → 2015년 탐지를 한 번에 푸는 YOLO(3부) → 2017년 Mask R-CNN, 2023년 Segment Anything(Meta)으로 아무 물체나 분할. 세 문제 모두 같은 다이얼 방식으로 푼다.

오늘의 자리

오늘은 **탐지**다. 1~3부에서 공개 모델로 상자를 받아 보고, 4부에서 **상자 그리기(정답 만들기)**를 직접 해 본 뒤 내 모델을 학습시킨다. 분할은 같은 YOLO의 `-seg` 모델로 바꾸기만 하면 된다(실습 3b).

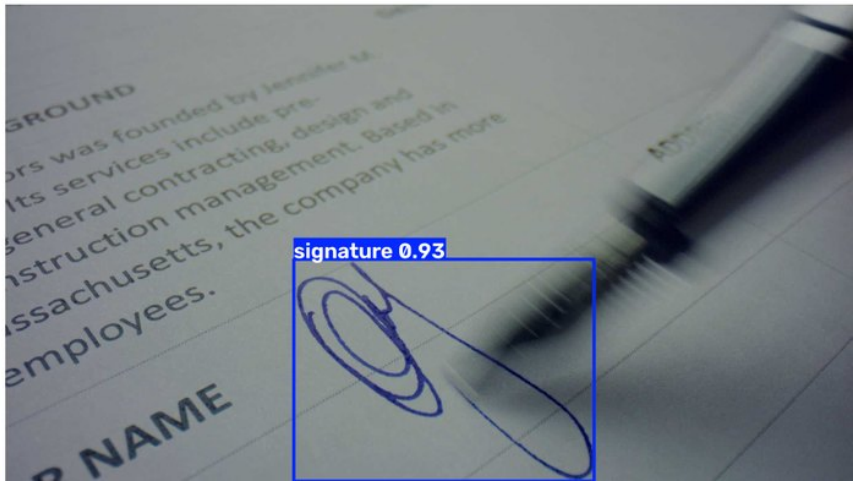
휴대폰에 이미 셋이 다 들어 있다. 사진 앱이 "음식 · 풍경"으로 묶는 것은 분류, 카메라가 얼굴에 네모를 그리는 것은 탐지, 인물 모드로 배경만 흐리는 것은 분할이다.

오늘의 결과물: 서명을 찾는 내 모델

학습 전 yolo26n: 찾은 물체 1개



학습 후 best.pt: 서명 1개



완성 기준 3개

✓ 사진 · 영상에서 사람 · 자동차를 찾고 **센다**
✓ 서명 사진으로 **내 모델**을 학습시켰다
✓ 학습 전후를 **같은 사진**으로 비교했다(학습 전엔 "책"으로 오인)

소재: 계약서 서명

문서 사진 178장(Ultralytics 공개 데이터셋). "이 계약서에 서명이 있는가, 어디에 있는가"는 공개 모델의 80종 목록에 **없는** 질문이다. 그래서 직접 가르친다.

준비 원칙 (1~3차시와 동일)

개인 계정
Google 계정은 개인 이메일로 만든 것

공개 사진만
오늘 사진 · 영상은 전부 공개 자료. 내 사진은 인터넷 공개 사진으로

얼굴 · 신분증 금지
가족 사진 · 신분증 · 업무 문서는 올리지 않는다

PC 초기화
노트북은 Drive에 남는다. 학습한 모델은 수업 끝에 내려받는다

⚠ Colab은 외부 서버다

올린 사진은 Google 서버로 간다. 3차시와 같은 이유로 오늘 올리는 것은 **공개 사진 · 공개 데이터셋뿐**이다. makesense.ai는 사진을 서버로 보내지 않고 브라우저 안에서만 다루지만, 역시 공개 사진으로 연습한다.

✋ 5분 규칙

5분 넘게 막히면 손을 든다. 에러의 빨간 글씨는 지우지 말고 그대로 둔다.



1부 사진 속 물체 찾기

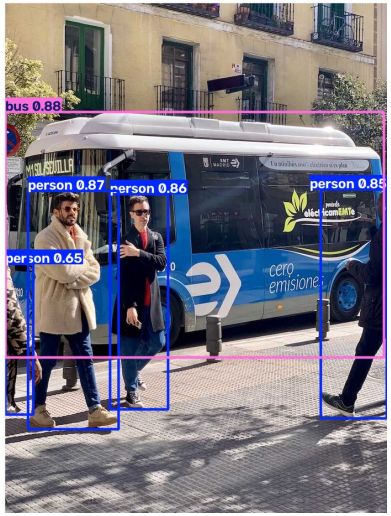
상자 · 이름 · 확신도

분류와 탐지: 무엇인가, 어디에 있는가

분류: 이 사진은 「버스」



탐지: 버스 1, 사람 4 + 위치



분류 (4차시)

사진 한 장에 이름표 **하나**. "이 사진은 버스다." 사람이 몇 명인지, 어디 있는지는 모른다.

탐지 (오늘)

물체마다 **상자 + 이름 + 확신도**. "버스 1대 여기, 사람 4명 저기." 세고, 위치를 알고, 영상에서 따라갈 수 있다.

탐지(detection)는 영어로 "찾아내기"다. 영상 속 표적 찾기 · 매장 손님 세기 · 불량품 골라내기가 전부 이 한 가지 능력이다.

탐지 결과는 세 가지 숫자다

상자 (box)
왼쪽 위 (x,y)와 오른쪽 아래 (x,y). 픽셀 단위. 이것이 "어디에"

이름 (class)
80종 목록 가운데 하나. 사람 · 자동차 · 버스 · 개 · 컵 … 이것이 "무엇"

확신도 (confidence)
0~100%. 모델이 "이 상자가 맞다"고 믿는 정도. 낮으면 버린다

>_ 노트북이 찍어 주는 표 (bus.jpg)

```
찾은 물체: 5개
  버스    확신도 88.1%  상자 왼쪽위(0,230) 오른쪽아래(803,751)
  사람    확신도 87.2%  상자 왼쪽위(49,398) 오른쪽아래(240,902)
  사람    확신도 65.4%  상자 왼쪽위(0,553) 오른쪽아래(60,871)
```

읽는 법

마지막 줄의 사람(65%)은 사진 왼쪽 끝에 **반쯤 잘려** 있다. 모델도 덜 확신한다. 확신도는 "맞을 확률"이라기보다 **버릴지 말지 정하는 손잡이**로 쓴다.

공개 모델이 아는 80종

묶음	이름 (한국어 표기는 노트북의 표와 같다)
사람 · 탈것	사람 · 자전거 · 자동차 · 오토바이 · 비행기 · 버스 · 기차 · 트럭 · 배
거리	신호등 · 소화전 · 정지 표지판 · 주차 미터기 · 벤치
동물	새 · 고양이 · 개 · 말 · 양 · 소 · 코끼리 · 곰 · 얼룩말 · 기린
소지품 · 운동	배낭 · 우산 · 핸드백 · 넥타이 · 여행가방 · 프리스비 · 스키 · 공 · 연 · 야구 방망이 · 스케이트보드 …
식탁	병 · 와인잔 · 컵 · 포크 · 칼 · 숟가락 · 그릇 · 바나나 · 사과 · 샌드위치 · 피자 · 도넛 · 케이크 …
실내	의자 · 소파 · 화분 · 침대 · 식탁 · 변기 · TV · 노트북 · 마우스 · 리모컨 · 키보드 · 휴대폰 …

COCO: 사진 12만 장에 사람이 그린 상자 90만 개

공개 모델은 **COCO**라는 공개 데이터셋으로 학습됐다. 일상 사물 80종. 4차시의 ImageNet이 "무엇인가"용 데이터였다면 COCO는 "어디에 있는가"용 데이터다.

⚠ **목록에 없는 것은 못 찾는다**: 서명 · 도장 · 안전모 · 명함. 엉뚱한 이름을 붙이거나 아예 못 본다. 4부에서 직접 채운다.

노트북 사용 규칙 3가지

1. 위에서 아래로
셀은 순서대로 누른다. 건너뛰면 뒤 셀이 앞 셀의 결과를 못 찾는다

2. 끝난 뒤 다음 셀
셀 왼쪽 괄호 안 숫자가 바뀌면 끝난 것. 빙글 도는 동안은 기다린다

3. 표시된 줄만 바꾼다
`# <-- 바꿔 보세요`가 붙은 줄만 고친다. 따옴표는 지우지 않는다

오늘 누를 셀은 11개

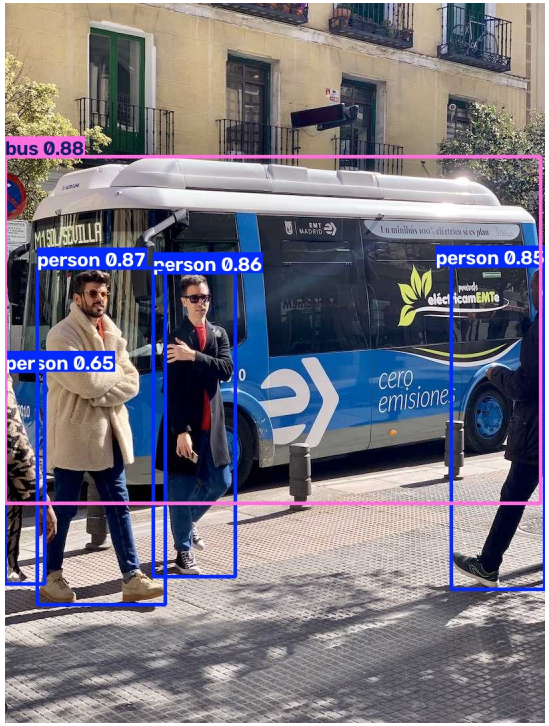
셀 1(준비)은 이미 눌렀다. 1부는 셀 2 · 3 · 4(사진), 2부는 셀 5 · 6(영상), 3부는 셀 7(모델 비교), 4부는 셀 8(학습) · 9(전후 비교) · 10(선택) · 11(내려받기). 셀마다 맨 위 주석에 번호와 할 일이 적혀 있다.

빨간 글씨가 나오면

에러는 지우지 말고 그대로 둔 채 손을 든다. 대부분은 **셀 1을 안 돌렸거나**(`NameError`) GPU 없이 돌려 느린 경우다. 셀 1부터 다시 누르면 대개 해결된다(안 될 때 표는 4부 끝).

실습 1: 사진 속 물체 찾기 (4분)

prompts/02_photo_detect.txt — 기대 출력과 체크



▶ 셀 2 ▶ — 이렇게 나오면 정상

찾은 물체: 5개 아래에 버스 1줄 · 사람 4줄, 처리 시간: 11 ms (한 장)(T4 실측), 그리고 상자가 그려진 사진. 처음 한 번은 모델을 메모리에 올리느라 몇 초 걸린다.

체크

✓ 표의 줄 수와 그림의 상자 수가 같다
✓ 확신도가 가장 높은 것이 표의 맨 위다
✓ 가장 낮은 사람(65%)이 사진 어디에 있는지 찾았다

한 장에 몇 ms

처리 시간이 **1초의 100분의 몇**이다. 이것이 영상(초당 30장)에도 쓸 수 있는 이유다. 2부에서 확인한다.

실습 2: 확신도 문턱값 · 특정 물체만 (5분)

prompts/03_threshold.txt — 바꿔 볼 값과 기록표

▶ 셀 3의 # <-- 바꿔 보세요 세 줄

```
CONF = 0.25      ONLY = ""      PHOTO = "bus.jpg"
```

1) PHOTO를 "crowd.jpg"(교차로 장면)로 바꾸고 ▶ **2)** CONF를 0.05로 → ▶ 0.9로 → ▶ **3)** CONF = 0.25, ONLY = "사람" → ▶

기록

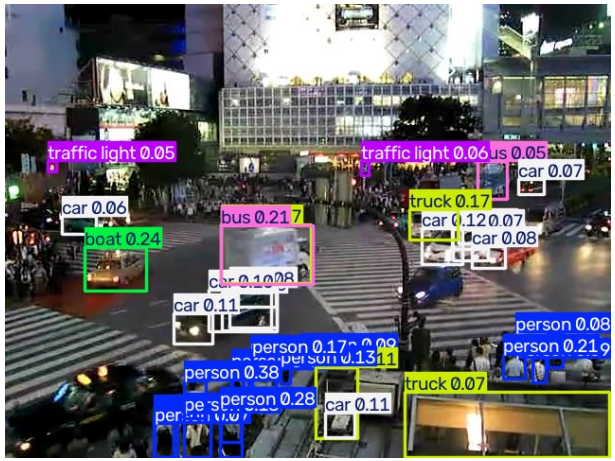
✓ 0.05일 때 찾은 수 ___개, 0.9일 때 ___개
✓ 0.05에서 **틀린 상자**(엉뚱한 이름)를 하나 찾았다
✓ "사람"만 남기면 ___명

자주 하는 실수

따옴표를 지우면 에러. "사람"처럼 큰따옴표 안에 쓴다. 80종에 없는 이름을 넣으면 노트북이 목록을 보여 준다.

문턱값을 올리면 틀린 상자가 줄지만 진짜도 놓친다. 내리면 그 반대. 정답은 없고 **용도가 정한다** — 다음 쪽.

문턱값이 하는 일: 놓치기와 잘못 잡기





↓ 문턱값 0.05: 37개

멀리 있는 사람까지 잡지만 **배 · 트럭 같은 엉뚱한 상자**도 섞인다. 놓치면 안 되는 일(안전 감시)에 가깝다.

↑ 문턱값 0.9: 0개

밤 거리의 작은 사람은 모델이 90% 확신하지 못한다. **틀리면 안 되는 일**(자동 결제)에는 이쪽이 맞다.

⚖ 두 종류의 실수

놓침(있는데 못 찾음)과 **잘못 잡음**(없는데 찾음)은 맞바꾸는 관계다. 문턱값 하나로 어느 쪽 실수를 줄일지 정한다. 4부의 "정확도"는 두 실수를 함께 세는 점수다.

실습 3: 내 사진 올려서 찾기 (5분)

`prompts/04_my_photo.txt` — 사진 고르는 법과 기록표

순서

1) 새 탭에서 이미지 검색(예: “사무실 회의”, “주차장”, “카페 테이블”) → 마음에 드는 사진을 **이미지를 다른 이름으로 저장**(바탕화면)
2) 셀 4 ▶ → **파일 선택** 버튼이 나오면 그 사진을 고른다 **3)** 표와 그림을 본다. 틀린 상자가 있으면 왜 틀렸을지 생각한다

기록

✓ 내 사진에서 찾은 물체 ___개
✓ 맞게 찾은 것 ___개, 틀린 것 ___개
✓ 80종에 없어서 **못 찾은 물체** 하나를 적었다

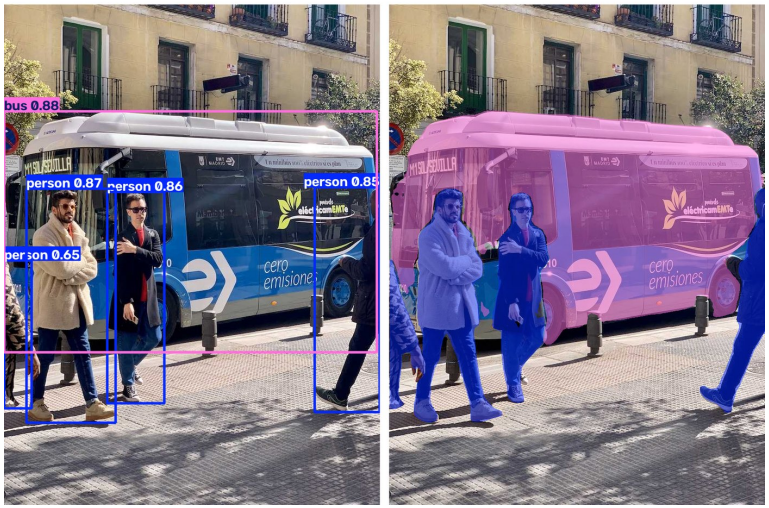
올리지 않는 사진

얼굴이 크게 나온 지인 사진 · 신분증 · 업무 문서. 파일 선택에서 **취소**를 누르면 샘플 사진(zidane.jpg)으로 진행된다.

잘 되는 사진과 안 되는 사진의 차이(밝기 · 크기 · 겹침 · 각도)가 2부 끝 “한계”의 내용이다. 지금 본 것을 기억해 둔다.

분할: 상자 대신 윤곽, 그리고 면적

탐지: 상자 분할: 윤곽



상자가 못 재는 것

상자는 물체 둘레의 **배경까지** 포함한다. "사진에서 버스가 몇 %인가", "병변이 몇 mm^2인가", "배경만 흐려라"는 픽셀 단위 윤곽이 있어야 답할 수 있다.

값은 라벨링에서 치른다

모델은 이름에 `-seg`만 붙이면 된다. 비싼 것은 **라벨링**이다. 상자 하나는 두 번 클릭, 윤곽은 점 수십 개. 그래서 오늘 4부의 서명 모델은 상자로 만든다.

셀 4b 실측(bus.jpg): 버스는 윤곽 29.2% vs 상자 46.9%, 사람 넷은 합쳐 윤곽 13.1% vs 상자 28.0%. 상자로 재면 버스가 사진의 절반이지만 실제로는 3할이다.

실습 3b: 분할 맛보기 (4분)

prompts/04b_segment.txt — 순서 · 기대 출력 · 기록표

순서

1) 셀 4b ▶ (셀 4 바로 아래). 셀 4에서 올린 내 사진에 분할 모델이 돈다 **2)** 왼쪽 상자 · 오른쪽 윤곽 그림, 표의 **윤곽 면적 %**와 **상자 면적 %**를 읽는다
3) `SEG_PHOTO = "bus.jpg"` 로 바꾸고 한 번 더 ▶ → 버스와 사람의 면적을 비교한다

기록

✓ 가장 큰 물체 ___: 윤곽 _% < 상자 _%
✓ 윤곽이 깔끔한 물체 · 지저분한 물체 하나씩
✓ 윤곽이 있어야 되는 일 하나

안 될 때

`NameError: model` → 셀 2를 먼저 ▶. "윤곽을 딸 물체를 못 찾았습니다" → 사진에 80종이 없다. `SEG_PHOTO`를 `"bus.jpg"`로.

왼쪽과 오른쪽은 **같은 사진, 같은 크기의 모델**이다. 다른 것은 정답의 모양(상자 / 윤곽)뿐이고, 그 정답을 만드는 품이 4부에서 상자를 고르는 이유다.

세 줄

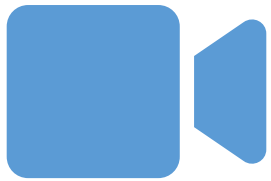
1) 분류는 “무엇”, 탐지는 “무엇이 **어디에 몇 개**”(상자 · 이름 · 확신도), 분할은 “**어디까지**”(윤곽 · 면적).
2) 공개 모델은 COCO 80종을 안다. 목록에 없는 것은 못 찾는다.
3) 확신도 문턱값은 **놓침**과 **잘못 잡음**을 맞바꾸는 손잡이다. 용도가 값을 정한다.

지금까지 한 것

셀 2 샘플 사진 → 셀 3 문턱값 · 사람만 → 셀 4 내 사진 → 셀 4b 분할(윤곽 · 면적). 노트북 안에 결과 그림이 남아 있다.

→ 다음

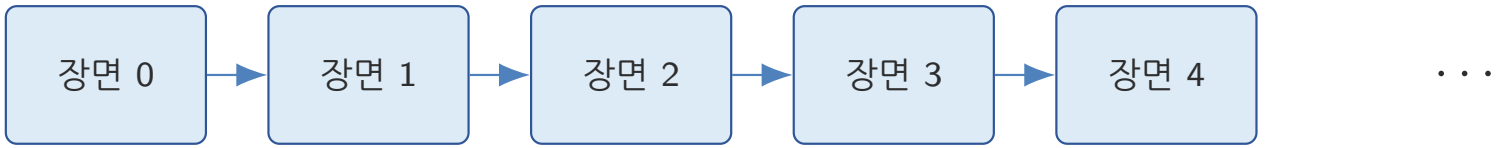
사진 한 장에 수십 ms면, 영상(초당 30장)도 된다. 2부에서 15초짜리 교차로 영상을 처리한다.



2부 영상 속 표적 찾기

영상은 사진의 연속 · 세기 · 쓰임과 한계

영상은 초당 30장의 사진이다



1초에 30장 → 15초면 450장. 한 장씩 탐지하고 다시 붙인다

영상 처리는 사진 처리의 반복

새로운 기술이 아니다. 1부의 셀 2를 450번 되풀이하고 결과 그림을 이어 붙이면 영상이 된다. 노트북의 셀 5가 정확히 그 일을 한다.

실시간이란

영상이 들어오는 속도(초당 30장)보다 처리가 빠르면 **실시간**. 한 장에 33 ms 이하여야 한다. YOLO가 유명해진 이유가 이 속도다 (3부).

오늘의 영상

도쿄 시부야 교차로(Wikimedia Commons, Emran Kassim, CC BY 2.0)의 앞 15초. 셀 1이 내려받아 `street.mp4`로 잘라 두었다. 사람 · 자동차 · 버스 · 트럭이 섞여 있다.

실습 4: 영상 속 물체 찾기 (6분)

`prompts/05_video.txt` — 기대 출력과 체크



▶ 셀 5 ▶ — 이렇게 나오면 정상

20~30초 뒤 두 줄: 처리한 장면: 450장 / 걸린 시간 7.7초 / 초당 58장 처리(T4 실측), 한 장면 속 사람 수: 최소 0명, 최대 7명, 평균 1.9명. 그 아래 **결과 영상**이 재생된다(상자가 따라다닌다).

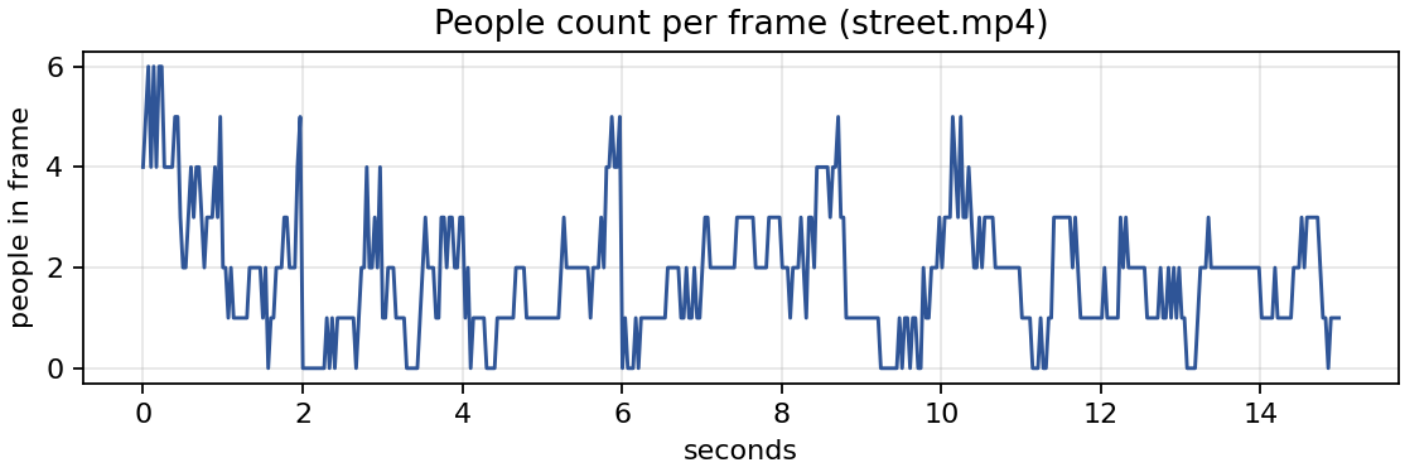
체크

✓ 초당 처리 장면 수가 ___장 — 영상의 30장보다 많은가(T4는 58장)
✓ 결과 영상에서 상자가 사람을 **따라가는지** 봤다
✓ 상자가 사라졌다 나타나는 순간을 하나 찾았다

⏳ **탐지는 7.7초인데 셀이 더 오래 도는 이유**: 상자를 그려 넣고 영상으로 다시 묶는 시간이다. CPU면 2~3배. 영상이 안 보이면 셀을 한 번 더 누른다.

실습 5: 장면마다 사람 수 세기 (3분)

prompts/05_video.txt — 그래프 읽는 법



▶ 셀 6 ▶

셀 5가 센 **장면별 사람 수**를 시간 순서로 그린다. 마지막 줄에 가장 붐빈 순간: N초, N명. 이 숫자가 결과 영상의 그 순간과 맞는지 본다.

세는 순간 데이터가 된다

상자 수를 세면 **숫자**가 되고, 숫자는 그래프 · 알림 · 보고서가 된다. 매장 혼잡도, 대기줄 길이, 빈 주차 칸 수가 전부 "상자 세기"다.

그래프가 들쭉날쭉한 것은 사람이 가려지거나 작아져 상자가 깜빡이기 때문이다. 실제 제품은 앞뒤 장면을 이어 붙여(추적) 이 떨림을 없앤다.

어디에 쓰나: 상자 세기 · 위치 알기 · 따라가기

세기
매장 방문객 수 · 대기줄 · 빈 주차 칸 · 선반의 남은 상품. 상자 수가 곧 숫자

있나 없나
안전모 미착용 · 출입금지 구역 진입 · 생산 라인의 불량품. 상자가 있으면 알림

따라가기
교차로 차량 흐름 · 매장 동선 · 스포츠 선수 이동. 장면마다의 상자를 이어 붙인다

세 가지 모두 같은 부품이다

탐지 모델 하나 + "상자로 무엇을 할지" 몇 줄. 셀 5 · 6에서 이미 "세기"를 했다. "있나 없나"는 `if 상자 수 > 0`, "따라가기"는 앞 장면의 상자와 짝 맞추기다. 어려운 부분은 모델이 아니라 **현장에 맞는 데이터**(4부)다.

질문 하나

내 일터에서 "몇 개인지 세는 일", "있는지 확인하는 일"을 하나씩 떠올려 본다. 그것이 탐지가 들어갈 자리다. 카메라가 이미 있다면 절반은 준비된 것이다.

한계: 가림, 작음, 어둠, 그리고 모르는 것

가림
사람 뒤의 사람. 상자가 하나로 합쳐지거나 사라진다

작음
멀리 있는 사람은 픽셀 몇 개. 모델이 보기엔 점이다

어둠 · 흔들림
밤 · 역광 · 비. 학습 사진에 드물었던 조건은 약하다

모르는 것
80종 밖의 물체. 못 보거나 엉뚱한 이름을 붙인다

고치는 법은 두 가지뿐

더 큰 모델(3부: n → m, 작은 물체를 더 잘 본다) 또는 **내 현장의 데이터로 다시 학습**(4부: 어둠 · 각도 · 새 물체). 둘 다 오늘 해 본다. 문턱값을 만지는 것은 임시방편이다.

실습 3에서 내 사진이 잘 안 됐다면 네 가지 중 어느 것이었는지 지금 말로 해 본다.

2부 요약

세 줄

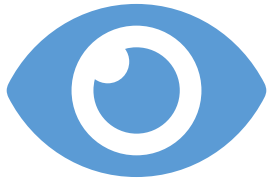
1) 영상 처리 = 사진 처리 × 장면 수. 초당 30장보다 빠르면 실시간.
2) 상자를 **세면 숫자**가 된다. 세기 · 있나 없나 · 따라가기가 탐지의 세 가지 쓰임.
3) 가림 · 작음 · 어둠 · 모르는 것이 약점. 고치는 법은 큰 모델 또는 내 데이터.

지금까지 한 것

셀 5 영상 450장 탐지 → 셀 6 사람 수 그래프. 결과 영상이 노트북 안에서 재생된다.

→ 다음

왜 이렇게 빠른가. 3부에서 YOLO라는 이름의 뜻과 "작은 모델 · 큰 모델"을 본다.



3부 YOLO는 어떻게

한 번만 본다 · 격자 · 크기와 속도

YOLO: You Only Look Once — 한 번만 본다

2015년 이전: 여러 번 본다

먼저 "물체가 있을 법한 자리"를 수천 군데 고른 뒤, 자리마다 분류기를 **따로** 돌렸다. 정확하지만 한 장에 몇 초. 영상에는 못 쓴다.

YOLO (2015): 한 번에 본다

사진 전체를 신경망에 **한 번** 통과시켜 모든 상자와 이름을 동시에 뱉는다. 한 장에 수십 ms. 그래서 영상이 된다.

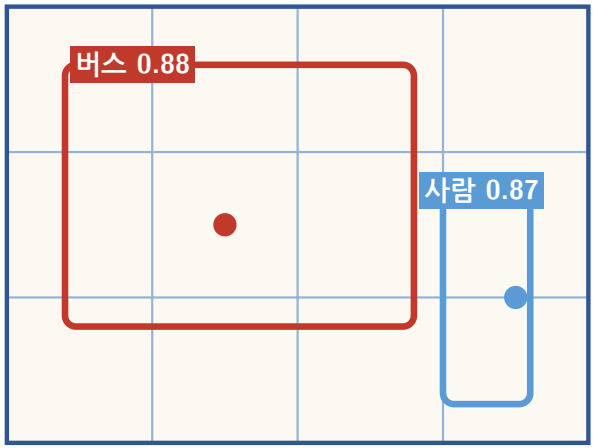
10년의 역사를 한 줄로

2015년 YOLO(v1) → v3(2018) → v5 · v8(Ultralytics) → YOLO11(2024) → **YOLO26(2026년 1월)**. 이름은 같고 속은 매번 바뀌었다. 오늘 쓰는 `yolo26n.pt`가 가장 최신이며, 뒤처리 단계를 없애 CPU에서도 빠르다.

4차시와 연결

"특징을 사람이 설계하지 않고 데이터가 정하게 했다"가 4차시의 결론이었다. YOLO는 거기에 "상자 찾기도 **한 신경망 안에서** 끝내자"를 더했다. 단계를 줄이면 빨라지고, 빨라지면 쓸 곳이 늘어난다.

격자로 나누고, 칸마다 답하게 한다



칸마다 "내 칸에 중심이 있는 물체는?"

한 번만 보는 방법

1) 사진을 격자(예: 20×20칸)로 나눈다
2) 칸마다 "내 칸에 **중심**이 있는 물체가 있나, 무엇인가, 상자는 어디까지인가"를 **동시에** 답하게 한다
3) 확신도가 낮은 칸의 답은 버린다(1부의 문턱값)

왜 빠른가

모든 칸이 **한 번의 계산**으로 같이 답한다. "자리마다 따로"가 없다. 대신 한 칸에 중심이 여러 개 몰리면(겹친 사람) 놓친다 — 2부의 "가림"이 바로 이것이다.

실제 YOLO는 큰 물체용 · 작은 물체용 격자를 여러 겹 쓴다. 원리는 위 한 겹과 같다.

크기와 속도: n · s · m · l · x

모델	다이얼(백만 개)	COCO 정확도(mAP)	느낌
yolo26**n** (nano)	2.4	40.9	가장 작고 빠름. 카메라 안 칩에도
yolo26**s** (small)	9.5	48.6	
yolo26**m** (medium)	20.4	53.1	오늘 비교용. 작은 물체에 강함
yolo26**l** (large)	24.8	55.0	
yolo26**x** (xlarge)	55.7	57.5	가장 정확. 서버 GPU용

수치는 Ultralytics 공식 문서(docs.ultralytics.com/models/yolo26, COCO val 기준). mAP는 높을수록 정확. 다이얼 수는 학습용 부품을 뗀 값이라 노트북 셀 7이 세는 수(n 2.6 · m 21.9)보다 조금 작다. T4 GPU 기준 한 장 처리는 n 1.7 ms · m 4.7 ms.

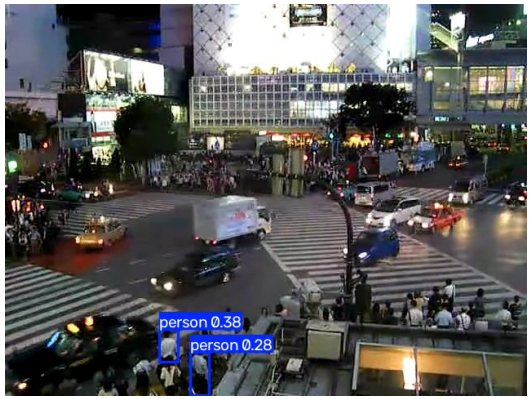
⚖ 같은 설계, 다른 크기

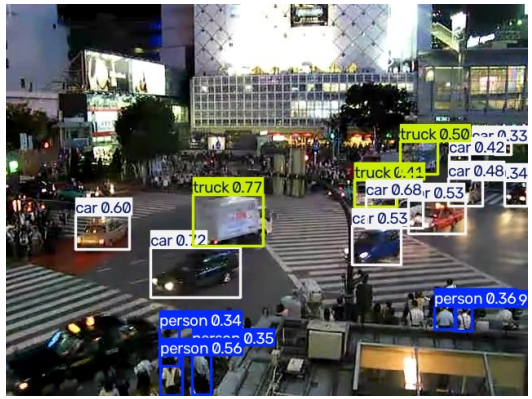
다섯 모델은 격자 방식이 같고 **다이얼 수**만 다르다. 4차시의 "규모를 키우면 좋아진다"가 여기서도 보인다. 대신 느려지고 메모리를 더 쓴다. 현장의 장비(카메라 안 칩인가, 서버 GPU인가)가 크기를 정한다.

💡 **고르는 법**: 먼저 n으로 시작한다. 작은 물체를 놓치면 한 단계씩 키운다. 내 데이터로 학습(4부)할 때도 n이 가장 빨리 끝난다.

실습 6: 작은 모델 vs 큰 모델 (5분)

prompts/06_n_vs_m.txt — 기록표





▶ **셀 7** ▶ (같은 사진 crowd.jpg, 같은 문턱값 0.25): 모델마다 다이얼 · 한 장 ms · 찾은 물체 한 줄과 그림, 끝에 비교 표. 첫 실행은 m을 올리느라 몇 초 더.

기록

✓ n: __ms · __개 m: __ms · __개
✓ m이 **더 찾은 것**이 무엇인지(멀리 있는 자동차 · 사람) 봤다

읽는 법

T4 실측: n 15.6 ms · 2개, m 32.1 ms · 16개. 다이얼 8배에 시간 2배, 찾은 물체 8배. 밤 거리의 작은 물체에서 차이가 크다. 낮의 큰 물체(bus.jpg)로 바꾸면 차이가 거의 없다.

3부 요약

세 줄

1) YOLO = 한 번만 본다. 사진을 격자로 나눠 칸마다 상자 · 이름 · 확신도를 **동시에** 답하게 한다.
2) 단계를 줄여 빨라졌고, 빨라져서 영상 · 카메라 · 현장에 들어갔다.
3) n부터 x까지 다이얼 수만 다르다. 작은 물체가 중요하면 키우고, 장비가 작으면 줄인다.

지금까지 한 것

셀 7에서 n과 m을 같은 사진에 돌려 시간과 찾은 수를 비교했다.

→ 다음

모델을 키워도 **80종 밖**은 못 찾는다. 4부에서 서명을 가르친다 — 오늘의 본론이다.



4부 내 데이터로 학습

라벨링 · 데이터셋 · 학습 · 전후 비교

서명은 80종에 없다

학습 전 yolo26n: 찾은 물체 0개



학습 후 best.pt: 서명 1개



학습 전

계약서 사진을 주면 **아무것도 못 찾거나** 엉뚱한 이름을 붙인다. 서명이라는 개념이 다이얼 어디에도 없다.

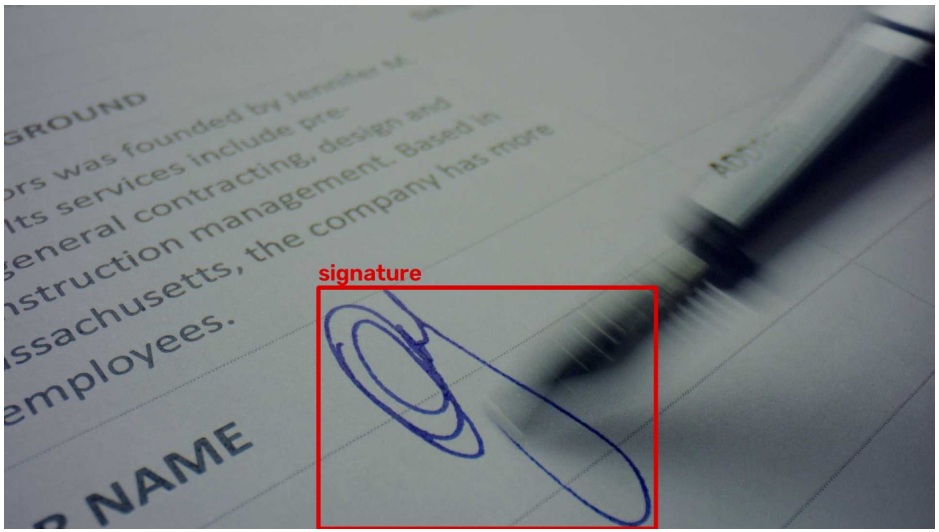
학습 후

서명 사진 143장 · 2분으로 다이얼을 **조금만** 다시 맞추면 찾는다. 처음부터가 아니라 **아는 것 위에 얹는다**(전이 학습).

왜 143장이면 되나

공개 모델은 이미 "선 · 모서리 · 글씨 질감"을 안다(4차시: 앞쪽 층은 보편적 특징). 새로 배울 것은 "이 뭉치를 서명이라 부른다"뿐이라 수백 장 · 몇 분이면 된다.

데이터셋 = 사진 + 좌표 파일



images/doc1.jpg

labels/doc1.txt 의 전부

```
0 0.519 0.770 0.363 0.460
```

0 = 이름 번호(signature는 0번) **0.519**
0.770 = 상자 중심 (가로, 세로) **0.363**
0.460 = 상자 폭, 높이

전부 사진 크기 대비 **비율**(0~1). 사진이 커지거나 작아져도 같은 줄이다.

폴더 구조

`images/`에 사진, `labels/`에 **같은 이름**의 txt. 사진당 txt 한 개, 물체당 한 줄. 이것이 YOLO 형식이다.

탐지의 "예시"는 **사진 + 사람이 그린 상자**다. 상자를 그리는 일이 **라벨링**이고, 실제 프로젝트 시간의 대부분이 여기 든다.

라벨링: 사람이 정답 상자를 그린다

딱 맞게
상자 가장자리가 물체에 닿게. 여백을 많이 두면 모델이 배경까지 서명으로 배운다

빠짐없이
한 사진에 있는 서명은 전부 그린다. 작거나 흐려도. 안 그린 것은 "서명 아님"으로 배운다

같은 기준으로
펜에 반쯤 가린 서명도 보이는 대로 그린다. 사진마다 기준이 흔들리면 모델도 흔들린다

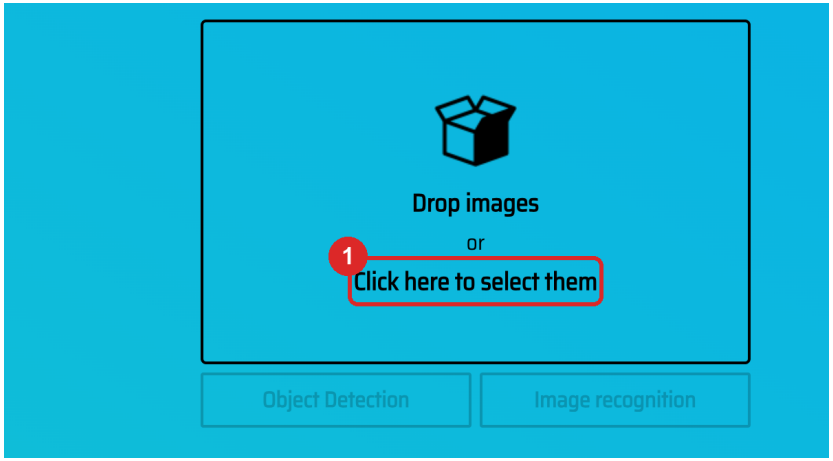
팁: 라벨링 가이드라인은 글로 적어 둔다

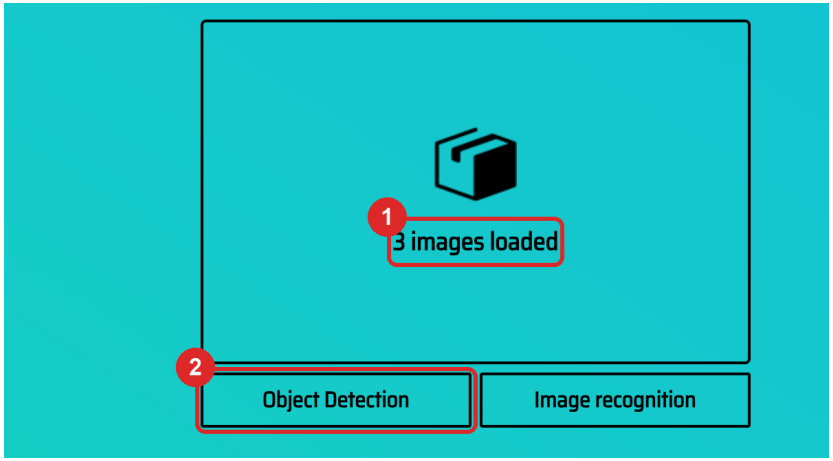
여러 사람이 나눠 그릴 때 가장 흔한 실패는 "기준이 다른 것"이다. **무엇을 그리고 무엇을 안 그리는지** 한 쪽짜리 글로 적고 예시 사진을 붙인다(Roboflow · Ultralytics 공식 가이드의 첫 번째 권고). 이름은 구체적으로("서명" > "표시").

얼마나 걸리나

상자 하나에 10~20초. 사진 200장이면 한 사람이 한두 시간. 실무에서는 먼저 조금만 라벨링해 모델을 만들고, 그 모델이 그린 상자를 **고치는 방식**으로 속도를 올린다.

makesense.ai 1: 사진 올리기





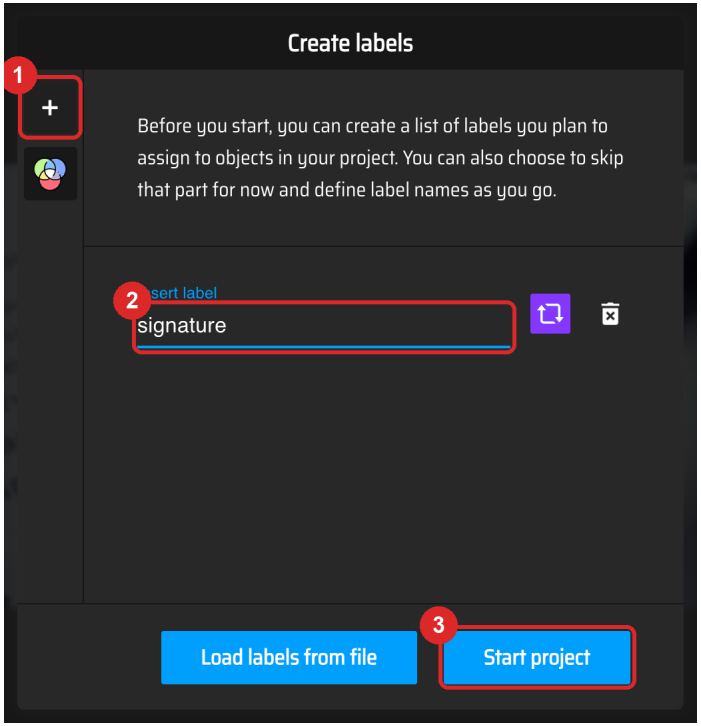
순서

1) 새 탭에서 `makesense.ai` → 오른쪽 아래 **Get Started** **2)** 왼쪽 **1** **Click here to select them** → `docs/라벨링연습/`의 `doc1~3.jpg` 세 장을 **한꺼번에** 선택

3) 오른쪽처럼 **1** **3 images loaded**가 보이면 **2** **Object Detection**

makesense.ai는 사진을 서버로 보내지 않고 브라우저 안에서만 동작한다(공개 소스). 탭을 닫으면 작업도 사라지므로 **끝나면 반드시 내보내기**한다.

makesense.ai 2: 라벨 이름 만들기



순서

1) 1 + 를 누르면 입력 칸이 생긴다
2) 2 칸에 `signature` 라고 쓴다(영문 소문자)
3) 3 **Start project**

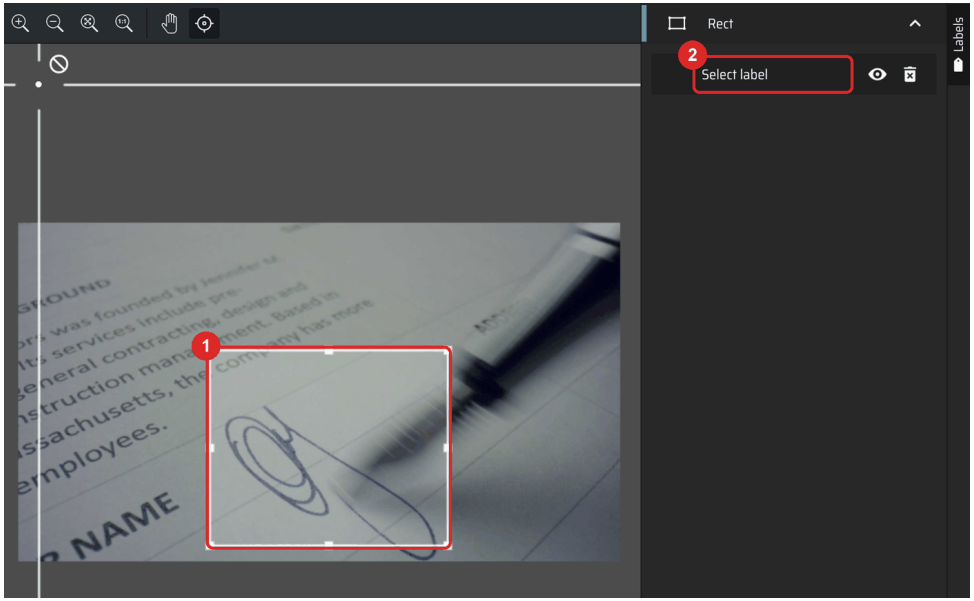
라벨 이름은 1부의 "이름"

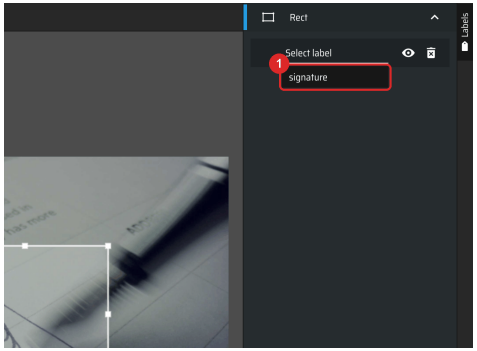
공개 모델의 80종 목록을 오늘은 **서명 한 종**으로 바꾸는 것이다. 이름이 둘 이상이면 여기서 +를 더 누른다(예: 서명 · 도장).

왜 영문인가

한글 이름도 되지만, 내보낸 파일을 다른 도구에서 읽을 때 글자가 깨지는 일이 있다. 이름은 영문, 설명은 한글로 두는 것이 관례다.

makesense.ai 3: 상자 그리고 이름 붙이기





순서

1) 사진 위에서 서명의 왼쪽 위에서 오른쪽 아래로 **드래그** 1

2) 오른쪽 2 **Select label** → **signature**

3) 상자가 보라색이 되면 끝. 아래 > 로 다음 사진

고치고 싶으면: 상자를 클릭해 모서리 점을 끈다. 오른쪽 목록의 휴지통으로 지운다. 펜에 가려져 있어도 **보이는 부분 전체**를 감싼다.

실습 7: 라벨링 체험 (10분)

prompts/07_labeling.txt — 순서 전문 · 정답 좌표 · 막힐 때

할 일

1) 세 사진 모두에 서명 상자를 그리고 signature를 붙인다(사진당 서명은 1개)
2) 위 메뉴 **Actions** → **Export Annotations** → **A .zip package containing files in YOLO format** 체크 → **Export**
3) 내려받은 zip을 열어 doc1.txt를 메모장으로 연다. 숫자 5개가 한 줄이면 성공

정답과 비교

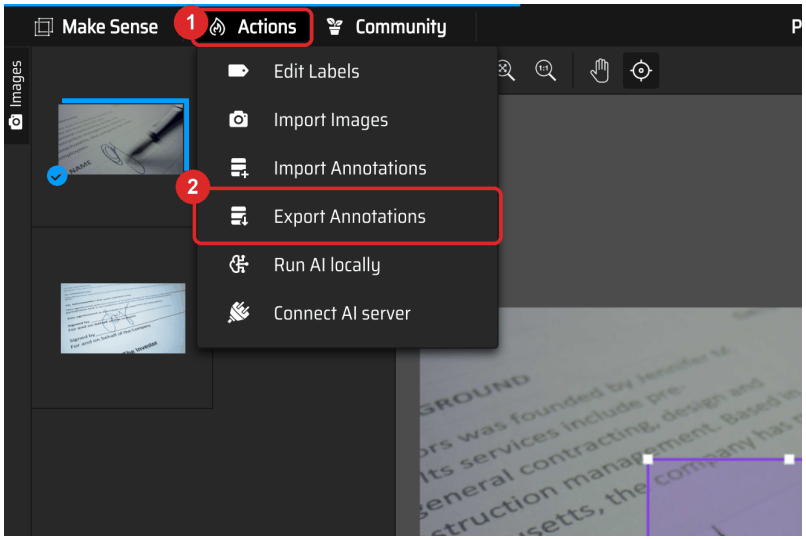
배포 폴더 docs/라벨링정답/doc1.txt에는 0 0.519 0.770 0.363 0.460. 내 숫자와 **둘째 자리까지** 비슷하면 잘 그린 것이다. 다르면 어느 쪽이 더 "딱 맞는지" 두 상자를 비교한다.

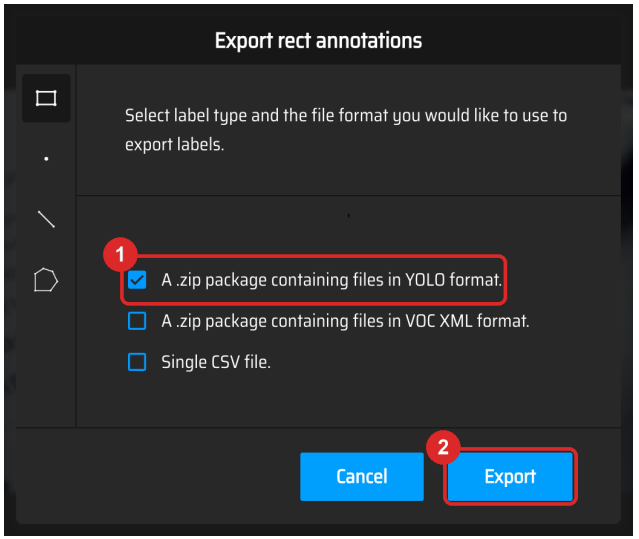
막힐 때

상자가 안 그려지면 위 도구줄의 **과녁 모양**(상자 그리기)이 켜져 있는지 본다. 드롭다운에 signature가 없으면 Start project 때 이름을 안 넣은 것 → Actions → Edit Labels. Export가 회색이면 YOLO 체크를 안 한 것.

오늘 학습에 쓰는 데이터셋 178장은 이 작업을 누군가가 178번 한 결과다. 내가 그린 3장은 거기에 더해도 되지만, 오늘은 시간 관계로 완성된 데이터셋으로 학습한다.

makesense.ai 4: 내보내기 화면





순서

왼쪽 1 **Actions** → 2 **Export Annotations** 오른쪽 1 **YOLO format** 체크 → 2 **Export**. 내려받기 폴더에 `labels_my-project-name_날짜.zip`이 생긴다.

zip 안에는 `doc1.txt doc2.txt doc3.txt` — 사진은 없고 좌표만 있다. 사진과 같은 폴더 구조에 넣으면 학습 데이터가 된다.

오늘의 데이터셋: 서명 탐지 178장

학습용 143장
모델이 보고 다이얼을 맞추는 사진. 상자 143개

검증용 35장
학습에 **쓰지 않고** 시험만 보는 사진. 정확도는 여기서 잰다

이름 1종
signature. Ultralytics 공개 데이터셋, 11MB, 셀 1이 내려받았다

왜 나누나: 외운 것과 배운 것

학습 사진으로 시험을 보면 **외운 것**도 맞는다. 안 본 사진 35장으로 시험을 봐야 **배운 것**이 보인다. 학습 사진만 잘 맞히고 새 사진은 못 맞히는 상태를 **과적합**이라 한다. 노트북의 "정확도"는 전부 검증용 35장의 점수다.

팁: 실무에서는 몇 장이 필요한가

Ultralytics 공식 권고: **시작은 수백 개**의 상자면 되고, 현장에 믿고 쓰려면 **종류당 1,500장 · 상자 10,000개** 이상. 오늘의 143장은 "되는지 보는" 규모다. 사진의 **다양성**(조명 · 각도 · 배경)이 장수보다 중요하다.

학습이란: 에포크와 손실

에포크(epoch)

학습 사진 143장을 **한 바퀴** 다 보는 것. 20 에포크 = 같은 사진을 20번 본다. 한 바퀴마다 다이얼이 조금씩 맞춰진다.

손실(loss)

"내가 그린 상자와 정답 상자가 얼마나 다른가"를 숫자 하나로. **낮을수록 좋다**. 학습은 이 숫자를 줄이는 방향으로 다이얼을 돌리는 일이다(4차시의 내리막).

셀 8이 보여 주는 것

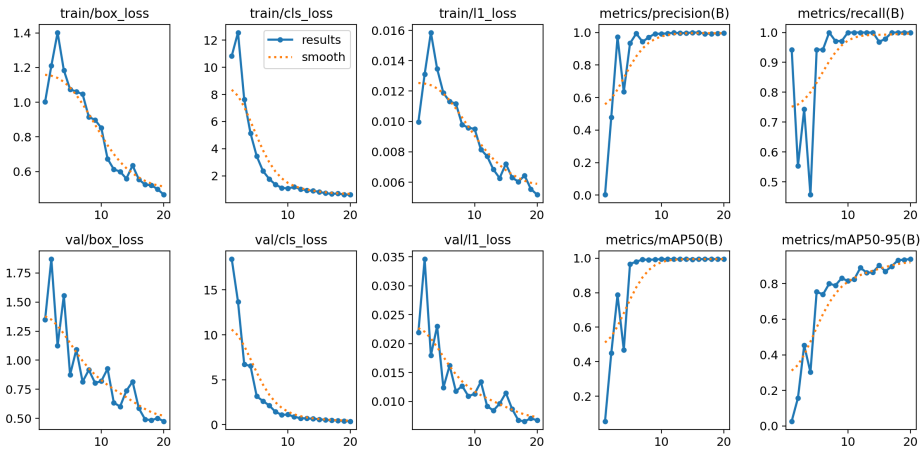
에포크마다 진행 막대 한 줄(손실 세 가지 · 사진 수). 끝에 검증 35장의 **mAP50** — "정답 상자와 절반 이상 겹치게 찾은 비율"로 읽으면 된다. 100%에 가까울수록 좋다. 그리고 `results.png` 그래프: 왼쪽 손실은 내려가고, 오른쪽 정확도는 올라가야 정상.

팁: 몇 에포크가 맞나

Ultralytics 공식 권고는 **300 에포크에서 시작**해 검증 정확도가 더 안 오르면 멈추는 것(`patience` 설정 = 조기 종료). 오늘은 시간 때문에 20. 실습 10에서 5 에포크와 비교해 "더 돌리면 좋아지나"를 직접 본다.

실습 8: 서명 탐지 모델 학습 (6분)

`prompts/08_train.txt` — 기대 출력과 기다리는 동안 볼 것



▶ 셀 8 ▶ 정상 출력

진행 막대가 1/20부터 20/20까지. 끝에 학습 시간: 1.5분, 검증 정확도(mAP50): 99.5%(T4 실측, 사람마다 조금 다르다), 그리고 그래프 한 장.

체크

✓ 왼쪽(loss)은 **내려가고** 오른쪽(mAP)은 **올라간다**

✓ mAP50 ___%, `best.pt` 경로를 봤다

기다리는 동안 (약 2분)

탭을 닫거나 새로 고치지 않는다. `box_loss`가 에포크마다 줄어드는지 본다. `GPU_mem`이 `0G`면 CPU — 멈추지 말고 끝까지 기다린다(10~15분). 그동안 다음 슬라이드를 듣는다.

학습 결과 읽기: 곡선 두 개

↓ 왼쪽: 손실이 내려간다

`train/box_loss`는 학습 사진에서의 실수, `val/box_loss`는 검증 사진에서의 실수. 둘 다 내려가면 정상. **train만 내려가고 val이 다시 올라가면** 과적합 — 외우기 시작한 것.

↑ 오른쪽: 정확도가 올라간다

`mAP50`은 "절반 이상 겹치면 맞음"으로 센 점수, `mAP50-95`는 더 엄격하게 센 점수. 앞 에포크에서 급히 오르고 뒤로 갈수록 완만해진다. 완만해진 지점이 "더 돌려도 소용 없는" 지점.

1부의 두 실수가 여기 있다

`precision`(잘못 잡지 않은 비율)과 `recall`(놓치지 않은 비율). 1부에서 문턱값으로 맞바꾸던 그 두 가지다. mAP는 문턱값을 여러 개 두고 두 실수를 함께 센 종합 점수다.

팁: 그래프가 이상할 때

손실이 안 내려가면 라벨이 틀렸거나(상자 어긋남) 데이터가 너무 적은 것. val이 다시 오르면 에포크를 줄이거나 사진을 더 모은다. 숫자보다 **전/후 사진 비교**(다음 실습)가 먼저다.

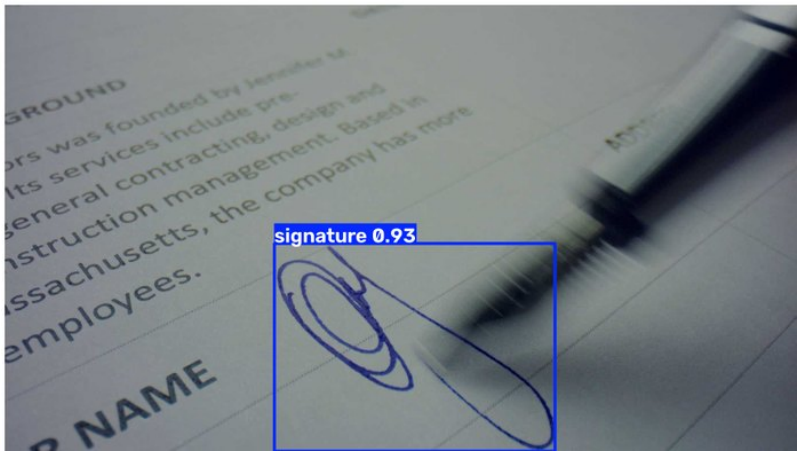
실습 9: 학습 전 · 후 비교 (6분)

prompts/09_before_after.txt — 바꿔 볼 사진 번호와 기록표

학습 전 yolo26n: 찾은 물체 1개



학습 후 best.pt: 서명 1개



▶ 셀 9 ▶

검증 사진 2장을 **학습 전 / 학습 후** 나란히, 각각 찾은 물체 표. 이어서 서명 영상(11초)을 처리해 재생한다. TEST_IMGS 번호(0~34)를 바꿔 다른 사진도 본다.

기록

✓ 학습 전: 찾은 물체 _개 (이름은?)
✓ 학습 후: 서명 _개, 확신도 __%
✓ 영상에서 서명 상자가 펜을 따라 **커지는지** 봤다

같은 사진, 같은 모델 구조, 바뀐 것은 다이얼 값뿐이다. 이것이 "내 데이터로 학습"의 전부다.

실습 10 (선택): 에포크를 줄이면 (3분)

prompts/10_epochs.txt — 비교표

▶ 셀 10 ▶ — 5 에포크만 학습한 모델과 비교

같은 데이터로 EPOCHS2 = 5만 돌린 모델의 mAP50을 셀 8의 20 에포크 모델과 나란히 찍는다 (약 30초). T4 실측: 5 에포크 92.6% vs 20 에포크 99.5%.

기록

✓ 20 에포크 ___% 5 에포크 ___%
✓ 차이가 큰가, 작은가. 그 이유를 한 줄로

읽는 법

5 에포크도 92%를 찾는다 — 공개 모델이 이미 많이 알기 때문. 더 돌리면 더 좋아지되 **체감이 줄어든다**. 시간 · 비용과 바꾸는 문제다.

시간이 남으면

셀 8의 EPOCHS를 40으로 바꿔 다시 돌려 본다(약 3분). 20과 40의 차이가 5와 20의 차이보다 작으면 "완만해지는 지점"을 본 것이다.

더 좋아지려면: 데이터, 시간, 크기

데이터를 더, 다양하게

가장 효과가 크다. 다른 조명 · 각도 · 종이 · 펜. 틀린 사진을 모아 라벨링해 넣는다

더 오래

에포크를 늘리되 검증 정확도가 멈추면 끝. 조기 종료(patience)를 켜 둔다

더 큰 모델

n → s → m. 작은 서명 · 흐린 사진에서 차이. 학습 시간도 같이 는다

팁: 공식 가이드가 꼽는 순서

Ultralytics "Tips for Model Training": **1)** 사전학습 모델에서 시작(오늘 한 것) **2)** 라벨은 빠짐없이 · 정확하게 **3)** 배경만 있는 사진(서명 없는 문서)도 1할쯤 넣으면 잘못 잡기가 준다 **4)** 에포크는 넉넉히, 조기 종료로 자른다 **5)** 그래도 안 되면 모델 크기. 데이터가 모델보다 먼저다.

4차시의 결론 그대로

"데이터와 계산을 늘리면 계속 좋아진다." 오늘의 세 손잡이가 정확히 데이터(사진) · 계산(에포크) · 규모(모델 크기)다.

저장과 이관: best.pt 한 파일

셀 11 ▶ — 내려받기

`best.pt`(약 6MB)가 내려받기 폴더에 온다. 이것이 오늘 학습한 **모델 전체**(다이얼 값 전부)다. USB나 Drive로 옮긴다. 실습실 PC는 초기화된다.

다른 PC에서 쓰기

어느 Colab에서든 `YOLO("best.pt")` 한 줄이면 1부의 셀 2와 똑같이 쓴다. 노트북 없는 PC에서는 Ultralytics 설치 후 같은 줄. 모델은 파일이고, 파일은 옮기면 된다.

내부에서 돌리려면

학습은 Colab(외부)에서 했지만 **쓰는 것은 내 PC**에서도 된다. 3차시의 Ollama와 같은 구조 — 모델 파일만 안으로 가져오면 사진은 밖으로 나가지 않는다. 다만 **학습 사진**은 Colab에 올렸으므로, 실제 업무 사진으로 학습하려면 내부 GPU가 필요하다.

한 줄 요약

탐지 프로젝트의 산출물은 **데이터셋(사진 + txt)** 과 **best.pt** 두 가지다. 둘 다 파일이고, 둘 다 다시 쓸 수 있다.

안 될 때: 증상별 조치

화면에 뜬 말	원인	조치
`NameError: name 'model'`	셀 1 또는 셀 2를 안 돌림	셀 1 → 셀 2부터 다시 ▶
GPU 없음 (셀 1)	런타임 유형이 CPU	런타임 → 런타임 유형 변경 → T4 → 셀 1 다시
"런타임 연결이 해제"	유휴 90분 또는 한도	런타임 → 다시 연결 → 셀 1부터
셀 5 영상이 안 보임	큰 출력 표시 실패	셀 5를 한 번 더 ▶. 숫자 두 줄이 있으면 처리는 된 것
'사람'은 80종 목록에 없습니다	따옴표 · 띄어쓰기	목록의 이름을 그대로 복사해 넣는다
셀 8이 15분 넘게 돈다	CPU로 학습 중	끝까지 기다리거나 `EPOCHS = 5`로 줄여 다시
`FileNotFoundError: best.pt`	셀 8이 끝나기 전에 셀 9	셀 8의 마지막 줄을 확인한 뒤 셀 9
makesense Export가 회색	YOLO 형식을 체크 안 함	첫 번째 항목 체크 후 Export

표 전문과 더 많은 증상은 `prompts/11_stuck.txt`. 5분 규칙은 그대로다.

네 줄

1) 데이터셋 = 사진 + 좌표 txt. 좌표는 사람이 그린다(라벨링). 딱 맞게 · 빠짐없이 · 같은 기준으로.
2) 학습용과 검증용을 나눈다. 정확도는 **안 본 사진**으로 잰다.
3) 공개 모델 위에 얹으면(전이 학습) 143장 · 20 에포크 · 2분(T4)으로 새 물체를 찾는다.
4) 더 좋아지는 순서: 데이터 → 에포크 → 모델 크기. 산출물은 데이터셋과 best.pt.

지금까지 한 것

makesense.ai 라벨링 3장 → 셀 8 학습 → 셀 9 전후 비교 → (셀 10) → 셀 11 best.pt 내려받기.

다음

정리. 오늘 배운 것을 여섯 줄로, 집에서 이어하는 법, PC 정리.

오늘의 핵심 정리

5차시에서 배운 것

1. **탐지 = 무엇이 어디에 몇 개.** 결과는 상자 · 이름 · 확신도. 공개 모델은 COCO 80종을 안다
2. **확신도 문턱값**은 놓침과 잘못 잡음을 맞바꾸는 손잡이. 용도가 값을 정한다
3. **영상 = 사진의 연속.** 한 장에 수십 ms라서 초당 30장을 따라간다. 상자를 세면 숫자(혼잡도 · 대기줄 · 빈 칸)가 된다
4. **YOLO = 한 번만 본다.** 격자 칸마다 동시에 답한다. n부터 x까지 다이얼 수만 다르다
5. **내 데이터로 학습**: 사진 + 상자 txt(라벨링) → 공개 모델 위에 얹어 몇 분 → 안 본 사진으로 검증. 학습 전엔 못 찾고 후엔 찾는다
6. **더 좋아지는 순서**: 데이터(다양하게) → 에포크(조기 종료) → 모델 크기. 산출물은 데이터셋과 best.pt 두 파일

★ Big Picture

4차시 "**예시를 보여 주면 다이얼이 맞춰진다**"를 오늘 손으로 했다.
예시(사진 + 상자)를 만들고, 다이얼을 맞추고(학습), 안 본 사진으로 확인했다. 글이든 사진이든 같은 순서다.

집에서 이어하기

prompts/00_README.txt — 실습 파일 목록과 공통 규칙, prompts/12_home.txt — 집에서 하는 순서

1. 내 물체 하나

80종에 없는 물체(명함 · 머그컵 · 특정 로고)를 정해 사진 30장을 찍는다. 조명 · 각도를 바꿔 가며

2. 라벨링 30장

makesense.ai로 상자를 그리고 YOLO 형식으로 내보낸다. 25장 학습용, 5장 검증용으로 나눈다

3. 학습 · 비교

12_home.txt의 폴더 구조대로 Colab에 올리고 셀 8을 돌린다. 검증 5장의 전후 비교가 결과물

한 바퀴의 완료 기준

검증 5장에서 학습 전 모델은 못 찾고 학습 후 모델은 찾는 그림 5장, 그리고 mAP50 숫자 하나. 잘 안 되면 "가림 · 작음 · 어둠 · 모르는 것" 중 무엇인지 적고 사진 10장을 더 찍어 다시 돌린다.

모든 실습은 개인 계정 · 공개 또는 직접 찍은 사물 사진으로. 사람 얼굴 · 업무 문서는 쓰지 않는다.

PC 정리 (공용 PC에서 나가기)

나가기 전 다섯 줄

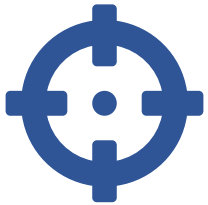
1) 셀 11로 `best.pt`를 내려받고 USB · Drive로 옮겼다 (노트북 자체는 Drive에 자동 저장)
2) makesense.ai 탭에서 내보낸 zip도 같이 옮겼다
3) Colab 메뉴 **런타임** → **런타임 연결 해제 및 삭제** (GPU 반납)
4) Google 로그아웃 (오른쪽 위 프로필 → 로그아웃), 브라우저 "비밀번호 저장"은 거절했는지 확인
5) 내려받기 폴더와 바탕화면의 사진 · zip · pt 파일 삭제

Drive에 남는 것

노트북(코드와 출력 그림). 학습한 모델 `best.pt`는 Colab 서버 쪽에 있어 **세션이 끝나면 사라진다** — 그래서 셀 11이 있다.

GPU 한도

무료 Colab은 GPU 사용 시간에 한도가 있다. 쓰지 않을 때 연결을 끊어 두면 다음에 받기 쉽다.



군-학 연계 프로그램 범용 교육

5차시 컴퓨터 비전 — YOLO 객체 탐지, 영상 속 표적 찾기

서왕덕 경기대학교 AI컴퓨터공학부